



Bases de Datos II

Trabajo Práctico Integrador: Etapa 2

Esta etapa consistirá en proveer un nuevo servicio de persistencia utilizando Spring Data JPA en lugar de usar Hibernate directamente, como se hizo en el primer trabajo práctico.

Para lograr esto, deberá crearse una nueva implementación de la interfaz **ToursService** llamada **ToursServiceImpl**, que deberá utilizar repositorios de Spring Data para resolver cualquier acceso a datos requerido.

En este caso se debe crear un repositorio por clase a persistir. Los repositorios implementados con Spring Data no son clases sino interfaces, y deberán extender `CrudRepository` definiendo los argumentos necesarios: tipo de entidad y tipo de id. Por ejemplo:

```
public interface RouteRepository extends CrudRepository<Route, Long>
```

Las consultas se pueden declarar utilizando query methods, los cuales serán implementados automáticamente por Spring Data (por ejemplo: `findBy<Propiedad>`), o mediante HQL utilizando la annotation `@Query` (notar que, dado que se trata de una interfaz, los métodos no pueden tener implementación).

Las clases de tests, `ToursApplicationTests` y `ToursQueryTests` validan, al igual que en la etapa previa, que la implementación de **ToursServiceImpl** cumpla con la funcionalidad básica y con el conjunto de consultas requeridas..

Para configurar SpringData se utiliza la clase `SpringDataConfiguration`, que deberán editar con sus credenciales de acceso a la BD y su nro de grupo.

La infraestructura del proyecto provista se encuentra en el mismo repositorio del TP previo, en la rama [tp2-springdata](#):

En concreto, se han modificado: la clase de configuración `AppConfig`, se ha reemplazado `HibernateConfiguration` con la clase de configuración `SpringDataConfiguration` y se han actualizado las dependencias necesarias en `pom.xml`; `DBInitializer` y por supuesto, los archivos de Testing.

Al igual que en la etapa anterior, deberán obtener un build exitoso en donde pasen todos los tests. Recordar que esto es condición necesaria pero no suficiente para aprobar la entrega del TP.



A tener en cuenta

Notar que en algunos casos, donde haya que retornar un número fijo de resultados y la consulta no pueda realizarse mediante query methods, el filtrado se debe realizar en el servicio. Para esto, se puede utilizar la interfaz Pageable de la siguiente manera:

- En la interfaz/repositorio, agregar un argumento al método de tipo Pageable
- En el servicio, crear un objeto de la clase PageRequest (que implementa Pageable) indicando el número de resultados deseados y enviarlo como argumento al mensaje al repositorio que realiza la consulta

Listado de consultas de esta etapa (se agregan nuevas a las ya realizadas)

- List<Purchase> getAllPurchasesOfUsername(String username)
Obtiene y retorna el listado de compras realizada por el usuario con el nombre de usuario especificado por parámetro.
- List<User> getUserSpendingMoreThan(float mount);
Obtiene y retorna el listado de usuario que hayan efectuado alguna compra por un valor mayor o igual al pasado por parámetro.
- List<User> getUsersWithNumberOfPurchases(int number);
Obtiene y retorna el listado de usuario que hayan efectuado al menos una cantidad de compras mayor o igual al valor pasado por parámetro.
- List<Supplier> getTopNSuppliersInPurchases(int n);
Obtiene y retorna los n proveedores (especificado por parámetro) con mayor cantidad de apariciones en compras.
- List<Supplier> getTopNSuppliersItemsSold(int n);
Obtiene y retorna los n proveedores (especificado por parámetro) con mayor cantidad de ítems vendidos. Tener en cuenta que la cantidad.
- List<Purchase> getTop10MoreExpensivePurchasesWithServices();
Obtiene y retorna las 10 compras de mayor precio total y que incluyen servicios.
- List<User> getTop5UsersMorePurchases();
Obtiene y retorna los 5 usuarios con mayor cantidad de compras registradas.
- List<Route> getTop3RoutesWithMoreStops();
Obtiene y retorna las 3 rutas con mayor cantidad de paradas.
- Long getCountOfPurchasesBetweenDates (Date start, Date end);
Obtiene y retorna la cantidad de compras registradas entre dos fechas.
- List<Route> getRoutesWithStop(Stop stop);
Obtiene y retorna el listado de rutas que incluyen una parada stop especificado por parámetro.



- `List<Purchase> getPurchaseWithService(Service service);`
Obtiene y retorna el listado de compras que incluyen el servicio especificado por parámetro.
- `Long getMaxStopOfRoutes();`
Obtiene y retorna la cantidad de paradas que posee el recorrido con mayor cantidad de stops.
- `Long getMaxServicesOfSupplier();`
Obtiene y retorna la cantidad de servicios que posee el proveedor con mayor cantidad de servicios.
- `List<Route> getRoutesNotSell();`
Obtiene y retorna el listado de recorridos que no fueron vendidas.
- `List<Route> getTop3RoutesWithMaxAverageRating();`
Obtiene y retorna los tres recorridos con mayor promedio de rating en las reviews de compras asociadas a dicha ruta.
- `List<Route> getRoutesWithMinRating();`
Obtiene y retorna los recorridos que tengan al menos una calificación de una estrella (rating)
- `Service getMostDemandedService();`
Obtiene y retorna el servicio que más veces fue incluido en compras, teniendo en cuenta la cantidad.
- `Route getMostBestSellingRoute();`
Obtiene y retorna la ruta que más veces aparece en compras, es decir, que más tickets vendió.
- `List<Service> getServiceNoAddedToPurchases();`
Obtiene y retorna el listado de servicios que no fueron incluidos en ninguna compra.
- `List<TourGuideUser> getTourGuidesWithRating1();`
Obtiene y retorna el listado de guías asignados a algún tour con un rating de una estrella.
- `DriverUser getDriverUserWithMoreRoutes();`
Obtiene y retorna el usuario de tipo DriverUser que posee más rutas asociadas

Fecha de entrega para esta etapa

Este último trabajo práctico tendrá como fecha de entrega máxima el **viernes 9 de mayo**.